

Introduction to Machine Learning

Automated Learning

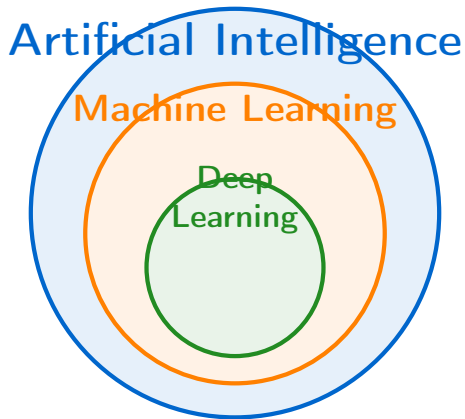
Meyssem Bouzaïen

Academic Year: 2025 - 2026

Outline

- 1 What is Machine Learning?
- 2 The Machine Learning Process
- 3 Types of Learning
- 4 Key ML Concepts
- 5 Practical Examples
- 6 ML Tools and Ecosystem
- 7 Best Practices and Advice
- 8 Conclusion

Reminder: AI vs Machine Learning



Where ML Fits Machine Learning is the branch of AI that enables machines to learn from data

What is Machine Learning?

Definition (Machine Learning)

Machine Learning is the study of computer algorithms that improve automatically through experience and the use of data.

Traditional Approach

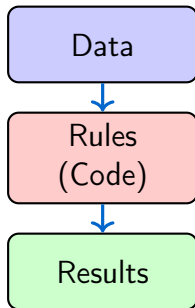
- Programmer writes the rules
- Explicit logic
- Difficult for complex tasks
- Example: if-then-else

Machine Learning Approach

- Machine learns the rules
- Learning from data
- Suited to complex tasks
- Example: image recognition

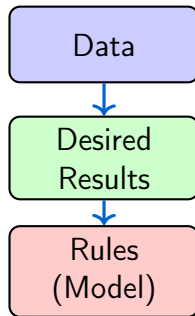
Classical Programming vs Machine Learning

Classical Programming



The programmer writes the rules

Machine Learning



The machine discovers the rules

Why Machine Learning?

Problems That Are Hard to Program

Some tasks are too complex to be programmed with explicit rules

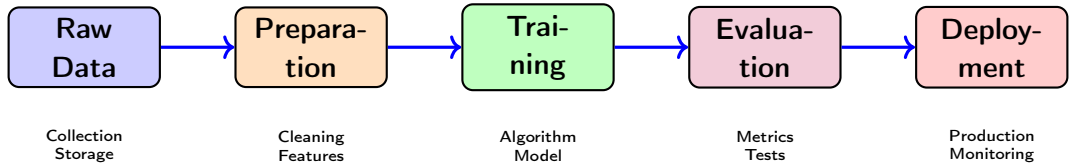
Examples of ML Tasks

- Facial recognition
- Spam filtering
- Machine translation
- Price prediction
- Product recommendation
- Medical diagnosis

Why Is It Difficult?

- Too many variables
- Complex patterns
- Non-obvious rules
- Data that evolves over time
- Numerous edge cases
- Context matters

The Machine Learning Pipeline



An Iterative Process

ML is a cycle: we often go back to previous steps to improve the model

Step 1: Data - The Fuel of ML

Fundamental Principle

No data = No Machine Learning

Data types: Structured (tables),
Unstructured (text, images), Time series,
Mixed

Data sources: Databases, Files (CSV,
JSON), APIs, Web scraping, IoT sensors

Data quality: Completeness, Accuracy,
Consistency, Relevance

Golden Rule

Garbage In, Garbage Out — Poor-quality
data → Poor-quality model

Step 2: Data Preparation (1/2)

Data Cleaning

A crucial step that can take up 60-80% of an ML project's time!

Main operations:

① Handling missing values

- Dropping rows
- Imputation (mean, median, mode)
- Predicting missing values

② Handling outliers

- Statistical detection
- Removal or correction

③ Handling duplicates

- Identification
- Removal

Step 2: Data Preparation (2/2)

Transformations:

4 Normalization / Standardization

- Bringing variables to the same scale
- Example: rescaling all values between 0 and 1

5 Encoding categorical variables

- Turning text into numbers
- Example: Male $\rightarrow$ 0, Female $\rightarrow$ 1

6 Feature Engineering

- Creating new variables
- Combining existing variables
- Example: Age computed from Date of Birth

Important

The quality of the preparation determines the quality of the final model

ML Vocabulary: Features and Labels

Example dataset: real estate price prediction

Surface	Bedrooms	Neighborhood	Price
120 m ²	3	Downtown	250k
80 m ²	2	Suburbs	180k



Features (X)
Input variables
Characteristics

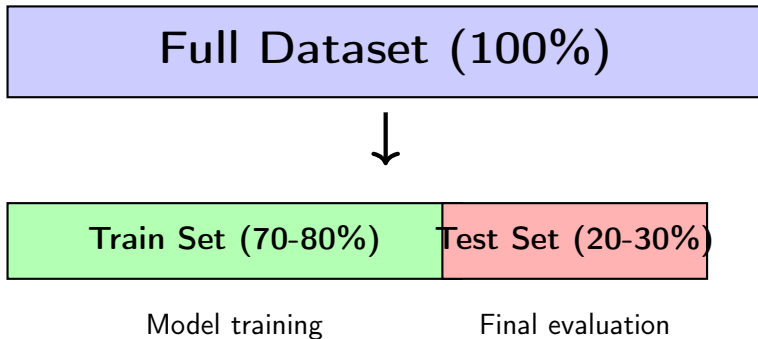


Label (Y)
Target variable
To predict

Step 3: Splitting the Data

Principle

We split the data into 2 or 3 sets to train and test the model



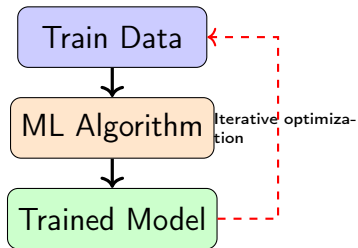
Important

The model must NEVER see the test data during training!

Step 4: Choosing and Training the Model

Choosing the algorithm

- Type of problem (classification, regression)
- Nature of the data
- Size of the dataset
- Need for interpretability
- Available resources



Training

- Feed the model with the training set
- The model adjusts its parameters
- Iterative optimization

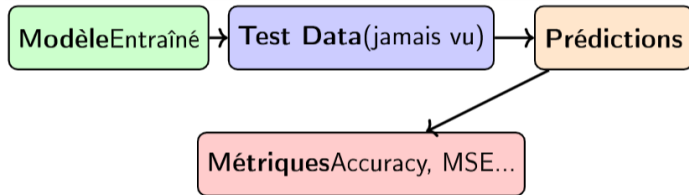
Result

A model capable of making predictions on new data

Step 5: Model Evaluation

Central Question

How do we know if our model is good?



Metrics by Problem Type

- **Classification:** Accuracy, Precision, Recall, F1-Score
- **Regression:** MSE, RMSE, MAE, R^2

The 3 Major Types of Learning

Supervised Learning

Labeled data

Classification / Regression

Unsupervised Learning

No labels

Clustering / Dim. reduction

Reinforcement Learning

Rewards / Penalties

Games / Robotics

Choosing the Type

The type of learning depends on:

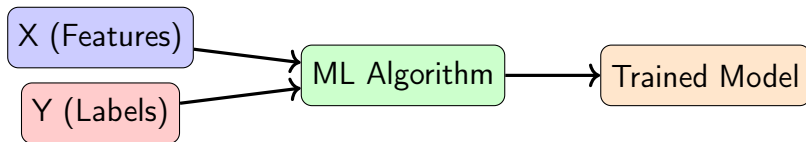
- Whether labels are available
- The project's goal
- The nature of the problem

Supervised Learning: Principle

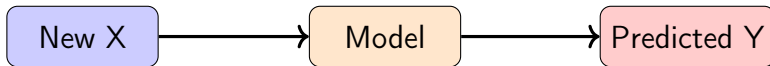
Definition

Learning from labeled examples to make predictions on new data

Training Phase



Prediction Phase



Supervised Learning: Examples

Classification

Email Spam

- **Input:** Email content
- **Output:** Spam or Not
- **Data:** 10,000 labeled emails

Regression

Real Estate Prices

- **Input:** Surface area, location
- **Output:** Price in €
- **Data:** Past sales

Medical Diagnosis

- **Input:** Symptoms, test results
- **Output:** Sick or Healthy
- **Data:** Medical records

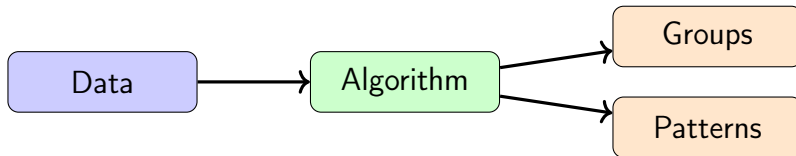
Future Sales

- **Input:** History, season
- **Output:** Sales volume
- **Data:** Historical data

Unsupervised Learning: Principle

Definition

Discovering hidden structures in data without labels



The algorithm discovers the interesting structures on its own

Unsupervised Learning: Examples

Clustering

Customer Segmentation

- **Input:** Age, income, purchases
- **Output:** 4 customer groups
- **Use:** Targeted marketing

Document Organization

- **Input:** Article texts
- **Output:** Similar topics
- **Use:** Automatic classification

Anomaly Detection

Bank Fraud

- **Input:** Transactions
- **Output:** Suspicious transactions
- **Use:** Security

Predictive Maintenance

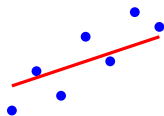
- **Input:** Sensor data
- **Output:** Abnormal behavior
- **Use:** Preventing failures

Overfitting vs Underfitting (1/2)

The Central Challenge of ML

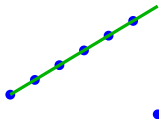
Finding the right balance between simplicity and complexity of the model

Underfitting



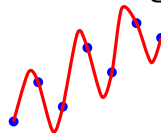
Too simple
Error on train
AND test

Good Fit



Balanced
Good generalization

Overfitting



Too complex
High error on test

Overfitting vs Underfitting (2/2)

Underfitting

- Model too simple
- Doesn't capture the patterns
- Poor performance everywhere

Solutions:

- More complex model
- More features
- Longer training

Overfitting

- Model too complex
- Memorizes the data
- Good on train, poor on test

Solutions:

- More data
- Regularization
- Simpler model
- Cross-validation

Goal

Build a model that **generalizes** well to new data

Bias and Variance: Definitions

Bias

Error due to the simplicity of the model

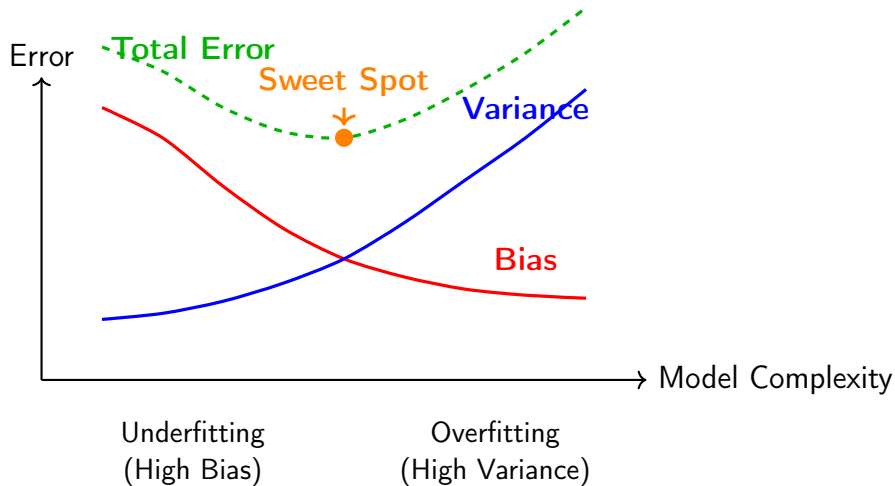
- The model makes overly simplistic assumptions
- Doesn't capture the true relationship in the data
- **High bias** → Underfitting
- **Example:** Using a straight line for non-linear data

Variance

Sensitivity of the model to variations in the data

- The model learns the noise in addition to the signal
- Very sensitive to the training data
- **High variance** → Overfitting
- **Example:** Degree-20 polynomial for 10 points

Bias-Variance Trade-off 1/2



Bias-Variance Trade-off 2/2

The Trade-off

- Reducing bias $\rightarrow$ increases variance
- Reducing variance $\rightarrow$ increases bias
- **Goal:** Find the optimal balance point

Cross-Validation (1/2)

Problem

How do we evaluate a model robustly when data is limited?

5-Fold Cross-Validation

Fold 1:	Test	Train	Train	Train	Train
Fold 2:	Train	Test	Train	Train	Train
Fold 3:	Train	Train	Test	Train	Train
Fold 4:	Train	Train	Train	Test	Train
Fold 5:	Train	Train	Train	Train	Test

 Training  Test

Cross-Validation (2/2)

Advantage

Every data point is used both for training and for testing → more reliable evaluation

Example 1: Email Classification (Spam)

The Problem

- Receiving thousands of emails
- Automatically identifying spam
- Avoiding false positives

The Data

- 10,000 labeled emails
- Spam: 40%
- Non-spam (Ham): 60%

Possible Features

- Specific words
- Message length
- Known/unknown sender

The Process

1 Preparation

- Text cleaning
- Feature extraction

2 Training

- Algorithm: SVM
- Train: 8,000 emails

3 Test

- Test: 2,000 emails
- Accuracy: 95%

4 Deployment

- Automatic filter
- Continuous improvement

Example 2: Price Prediction (Real Estate)

The Problem

- Estimating a house's price
- Supporting buy/sell decisions
- Regression problem

The Data

- 5,000 recent sales
- Prices from €50k to €500k

Features Used

- Surface area (m²)
- Number of bedrooms
- Location (GPS)
- Year built

The Process

- 1 Preparation
 - Feature normalization
 - Location encoding
- 2 Training
 - Algorithm: Linear Regression
 - Train: 4,000 houses
- 3 Test
 - Test: 1,000 houses
 - Average error: $\pm \text{€}15\text{k}$
- 4 Usage
 - Input: house characteristics
 - Output: estimated price

Example 3: Customer Segmentation (Clustering)

The Problem

- E-commerce with 50,000 customers
- Need for homogeneous groups
- Personalized marketing
- Unsupervised learning

The Data

- Purchase history
- Browsing behavior
- Demographic data

Features

- Amount spent
- Purchase frequency
- Preferred categories

The Process

- 1 Preparation
 - Normalization
 - Feature selection
- 2 Clustering
 - Algorithm: K-Means
 - Choice $K=4$ groups
- 3 Results
 - Group 1: VIP (5%)
 - Group 2: Regulars (35%)
 - Group 3: Occasional (45%)
 - Group 4: Inactive (15%)
- 4 Action
 - Personalized offers
 - Targeted campaigns

The Python Ecosystem for ML

Essential Libraries

NumPy

Numerical computation and multidimensional arrays

Pandas

Manipulation and analysis of tabular data

Matplotlib / Seaborn

Data visualization and plotting

Scikit-learn

Classical ML algorithms and preprocessing

Deep Learning Frameworks

TensorFlow

Google's framework for production

PyTorch

Meta's framework for research

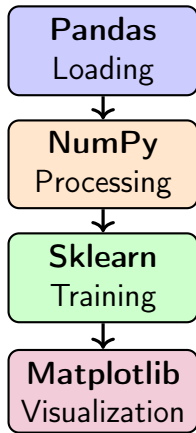
Keras

High-level API for simplicity

Development Environments

- Jupyter Notebook
- Google Colab
- VS Code / PyCharm

Typical Workflow with Python



```
import pandas as pd  
df = pd.read_csv('data.csv')
```

```
import numpy as np  
X = np.array(df[features])
```

```
from sklearn.tree import ...  
model.fit(X_train, y_train)
```

```
import matplotlib.pyplot as plt  
plt.plot(results)
```

The 10 Commandments of ML (1/2)

1 Start simple

- Simple baseline before a complex model
- Linear regression before deep learning

2 Understand the data

- Descriptive statistics
- Exploratory visualization

3 Rigorous cleaning

- Check for missing values
- Detect outliers

4 Feature Engineering

- Create relevant features
- Test different transformations

5 Proper validation

- Train/Test split
- Cross-validation

The 10 Commandments of ML (2/2)

6 Avoid overfitting

- Regularization
- More data

7 Choose the right metrics

- Based on the business problem
- Not just accuracy

8 Iterate and improve

- Test several algorithms
- Tune hyperparameters

9 Document the work

- Clear notebooks
- Reproducibility

10 Think about deployment

- Production-ready code
- Model monitoring

Common Pitfalls to Avoid

Data Errors

- Data leakage
- Features from the future
- Bias in the data
- Class imbalance

Modeling Errors

- Overfitting
- Underfitting
- Poor validation
- Ignoring outliers

Methodological Errors

- Testing on training data
- Optimizing on the test set
- Ignoring business context
- Features unavailable in production

Communication Errors

- Unrealistic promises
- Ignoring limitations
- No explanation of the model

Important

Most ML failures come from data problems, not algorithm problems!

What We Have Learned

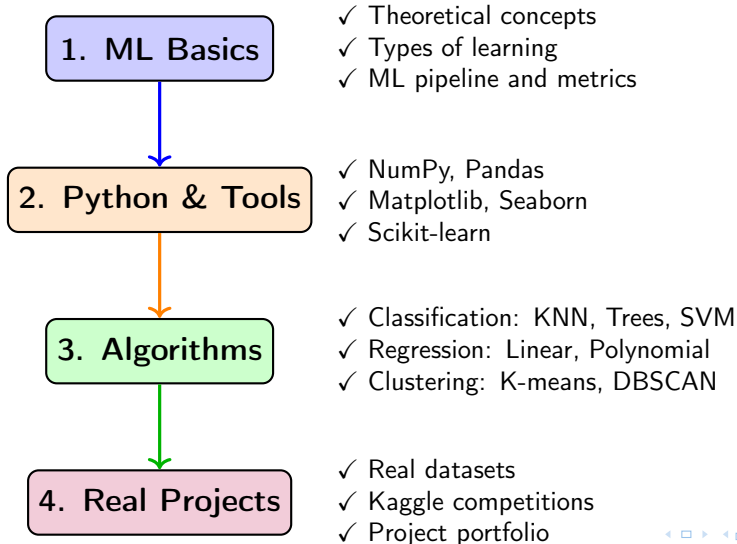
Fundamental Concepts

- Definition and principle of Machine Learning
- Difference from classical programming
- The complete ML pipeline
- Types of learning (supervised, unsupervised)

Key Concepts

- Features and Labels
- Train/Test split
- Overfitting and Underfitting
- Cross-validation
- Bias-Variance trade-off

The ML Learning Path



Next Step: Python for ML

Next Class

We will learn the essential Python tools for ML:

- NumPy: array manipulation
- Pandas: data analysis
- Matplotlib: visualization
- Scikit-learn: first ML models

Recommended Resources

- Install Anaconda (Python + libraries)
- Create a free Google Colab account
- Explore Kaggle for datasets